

Risk Management Dashboard

Data Flow — Architecture Overview

SBIT · Vedanta Limited · 2026

1. High-Level Overview

The Risk Management Dashboard ingests historical incident data from **Vedanta's SQL Server database** (OL_INCIDENTS), runs a four-step **ML pipeline** to compute risk scores, forecasts, and SHAP-based driver attributions, then serves all results through a **FastAPI backend** to a **React frontend**.

■ SQL Server — vedanta (Source of Truth)

- Table: OL_INCIDENTS · ~17,000 rows · 37 sites · 2020–2026
- Columns used: SINAME (site), BUNAME (business unit), YEAR, QUARTER, MONTH, LEVELNAME (severity), INCIDENTCATNAME (category), OCCUREDDATE, REPORTEDDATE
- Populated externally by Vedanta EHS system — this app never writes to OL_INCIDENTS
- All 6 ML output tables also live in the same SQL Server database

2. ML Pipeline (risk_scores → forecasters → backtest → drivers)

① Step 1 — Risk Scores (≈ 3 s)

- Reads all OL_INCIDENTS rows (2020+)
- Computes 4 sub-indices per site per quarter: Frequency · Severity · Velocity (QoQ growth) · Diversity (Shannon entropy)
- Composite score = $0.35 \cdot F + 0.30 \cdot S + 0.20 \cdot V + 0.15 \cdot D$ → scaled 0–100
- Risk level bands: Low ≤ 40 · Medium ≤ 65 · High ≤ 85 · Critical ≤ 100
- Output → risk_scores table (777 rows · 37 sites · 2021-Q1 to 2026-Q4)

▼ feeds predictions model

② Step 2 — Forecasters (≈ 66 s)

- Builds monthly incident time-series per site from OL_INCIDENTS
- Trains Prophet AND XGBoost on each site's series
- Champion model = whichever achieves lower holdout RMSE (last 3 months)
- When both succeed → ensemble average; sparse sites fall back to BU-level Prophet
- Predicts next 3 fiscal quarters (2026-Q4 · 2026-Q1 · 2026-Q2)
- Output → predictions_cache (111 rows) + model_runs training log

▼ evaluates forecast quality

③ Step 3 — Backtest (≈ 71 s)

- 6-month walk-forward holdout: train on data before cutoff, predict 6-month window
- Metrics: $\text{abs_pct_error} = |\text{actual} - \text{predicted}| / \text{actual} \times 100$
- MAPE ranges from ~22% (high-volume BALCO) to higher for sparse sites
- VLCTPP skipped — insufficient monthly history for holdout window
- Output → backtest_results (216 rows · 36 sites × 6 months)

▼ attributes incident categories

④ Step 4 — Drivers & Recommendations (≈ 58 s)

- SHAP values from XGBoost identify top-10 incident categories driving risk per site
- Sparkline: 6-month monthly counts per category (stored as JSON array)
- Rules engine fires 8 rules → RecommendationSpec objects (high_velocity · access_control · material_handling · ir_worker · asset_property · reporting_lag · process_deviations · generic_fallback)
- Output → risk_drivers (278 rows) + recommendations (47 rows)

3. Pipeline Triggers

Nightly (APScheduler)	Daily at 02:00 UTC · configurable via RETRAIN_CRON env var
CSV Upload	POST /api/ingest → on_success hook fires run_full_pipeline(trigger='post_ingest')
Manual	POST /api/admin/retrain or python scripts/run_pipeline.py --trigger manual

4. FastAPI Backend → Frontend

Endpoints (reads SQL Server)	Frontend Tabs (React + React Query)
- GET /api/sites · /api/kpis - GET /api/incidents/by-type · by-category · by-site · trend · heatmap - GET /api/risk-scores?site=&quarter;=&latest;_only= - GET /api/predictions?site= · /predictions/backtest - GET /api/drivers?site= · /recommendations?site= - GET /api/admin/freshness · /admin/runs - POST /api/ingest (CSV upload) - POST /api/admin/retrain (manual trigger)	- Overview — KPI cards + breakdown charts - Trends — incident count over time - Risk Drivers — SHAP bars + sparklines - Predictions — forecast + confidence bands - Recommendations — rules-based action items - AI Insights — placeholder stub - Reports — placeholder stub

5. CSV Upload Flow (only user-triggered write)

	User selects CSV in UI	Frontend
2	POST /api/ingest → temp file saved	FastAPI
	cleaner.py validates rows → quarantine bad dates / bad years	services/cleaner.py
4	ingestion_runs record created (status: running → success/failed)	SQL Server
	on_success hook fires → full pipeline runs in background (~3 min)	orchestrator.py
6	Dashboard refreshes → serves updated scores & forecasts	Frontend

6. Key Constraints & Design Decisions

OL_INCIDENTS	Read-only. Never written by this app. External Vedanta EHS system is the only writer.
--------------	---

Sole database	SQL Server vedanta. Postgres stack was retired 2026-05-25.
api/ + schemas/	Vinay's territory — API files and Pydantic schemas are not modified by the ML team.
Fiscal quarters	Q4 = Jan–Mar · Q1 = Apr–Jun · Q2 = Jul–Sep · Q3 = Oct–Dec
Site normalisation	OL_INCIDENTS has mixed-case site names. All normalised to UPPER STRIP before scoring.
Sparse-site fallback	Sites with <50 incidents or <12 months of data fall back to BU-level Prophet scaled by historical share.